Guía Integral de Arquitectura, División de Responsabilidades y Manual de Configuración

Proyecto CineTrack · Programación IV (UTN - FRRE) · Repositorio: Estefi y Ari

Esta es la guía integral de arquitectura, división de responsabilidades y manual de configuración paso a paso para que Ari prepare Antigravity IDE en su computadora y desarrolle su frente sin colisiones de código ni sobrecarga de almacenamiento.

1. Matriz de División de Responsabilidades (Estefi vs. Ari)

El desarrollo se divide desacoplando los servicios del monorepo en base a las dos líneas académicas de la cátedra de Programación IV:

Eje / TP	Frente de Estefi (Línea Python e Infraestructura)	Frente de Ari (Línea JavaScript y Gestión)
Rol Académico	Responsable de Integración: Controla la comunicación inter-servicios, la consistencia de contratos y la orquestación.	Responsable de Repositorio: Administra el tablero de tareas, audita ramas y revisa la calidad de los PRs.
TP 0: Fundación	Docker Compose local (Postgres y Mongo), setup de Django y especificación base en docs/tp-0-propuesta/.	Setup del tablero GitHub Projects con historias MoSCoW, revisión del PR inicial y setup de package.json.
TP 1: APIs CRUD	backend-django/: Modelos relacionales (Genre, MediaItem, Season, Episode, UserProgress), ViewSets, filtros, paginación y Pytest.	backend-node/: Schemas Mongoose (Review, replies anidadas), controladores Express, validación con Zod, Jest y cliente Axios a Django con fallback.
TP 2: Seguridad	backend-django/: Emisión de SimpleJWT, modelo CustomUser, hashing con sal, rate limiting y permisos de objeto.	backend-node/: Middleware de validación criptográfica de JWT compartido, protección de rutas y checklist OWASP.
TP 3: IA & Tiempo Real	service-ai/: FastAPI + Ollama (llama3.2:1b), esquemas Pydantic v2 y optimización relacional en Postgres (EXPLAIN ANALYZE).	backend-node/: Servidor Socket.IO 4 (WatchRooms, presencia), consumo del microservicio de IA para detección de spoilers.
TP 4: Despliegue	docker-compose.yml multietapa, pipeline de CI en GitHub Actions y documentación OpenAPI en Django.	Documentación Swagger en Express, Dockerfile de Node y endpoint GraphQL integrando datos de ambas bases.
TP 5: TFI Final	Despliegue integral reproducible en un comando, diagramas C4 y defensa del núcleo relacional/IA.	Redacción de planificación y retrospectiva en el informe PDF (20-35 págs) y demo de la capa en tiempo real.

2. Preparación del Entorno Local de Ari (Sin sobrecarga de disco)

Ari no necesita instalar Docker, PostgreSQL ni Ollama en su máquina. Su entorno solo requiere dependencias livianas:

Requisitos Previos a Instalar

- **Node.js 20 LTS:** Descargar e instalar desde nodejs.org (incluye npm).
- **Git:** Descargar e instalar desde git-scm.com.
- **Antigravity IDE:** Instalar y abrir en su equipo.

Conexión de Base de Datos Liviana (MongoDB Atlas)

Para no instalar MongoDB localmente ni consumir memoria RAM:

1 Ari crea una cuenta gratuita en MongoDB Atlas.

2 Crea un clúster gratuito compartido (Shared M0 - 512 MB).

3 En **Database Access**, crea un usuario y contraseña.

4 En **Network Access**, añade la IP `0.0.0.0/0` (permitir acceso desde cualquier lugar).

5 Copia la cadena de conexión URI (ej. `mongodb+srv://ari:<password>@cluster0.mongodb.net/cinetrack_social`).

Clonado y Configuración del Repositorio

Ari ejecuta en su terminal (Git Bash o PowerShell):

```
# 1. Clonar el repositorio
git clone https://github.com/BorghiEsteff/TPI-ProgIV.git
cd TPI-ProgIV

# 2. Configurar sus credenciales de Git
git config user.name "Nombre de Ari"
git config user.email "correo-de-ari-en-github@gmail.com"

# 3. Crear su rama de trabajo para el TP 1
git checkout -b feature/node-reviews-crud

# 4. Configurar el archivo de variables de entorno de Node
cd backend-node
cp .env.example .env
```

En su archivo `backend-node/.env`, Ari edita las variables:

```
PORT=5000
NODE_ENV=development

# Conexión a su MongoDB Atlas gratuito
MONGODB_URI=mongodb+srv://ari:TU_PASSWORD@cluster0.mongodb.net/cinetrack_social

# Clave idéntica a la que utiliza Django en la máquina de Estefi
JWT_SIGNING_KEY=super-secret-jwt-signing-key-shared-between-services

# URL de Django (para pruebas mockeadas o cuando Estefi le brinde acceso)
DJANGO_API_URL=http://localhost:8000/api/v1
DJANGO_TIMEOUT_MS=2500

CLIENT_ORIGIN=http://localhost:5173
```

Instala las dependencias de Node:

```
npm install
```

3. Configuración de Antigravity IDE en la Computadora de Ari

Para que el Antigravity de Ari trabaje de forma autónoma sin tocar el código de Python ni violar las políticas de Git del equipo:

Paso 1: Configurar las Reglas Locales del Agente

Ari debe crear o verificar el archivo `.antigravity/rules.md` en la raíz de su proyecto con el siguiente contenido:

```
# Antigravity Rules - Frente de Desarrollo Node.js (Ari)

## 1. Delimitación Estricta de Alcance (Scope)
- Tu ámbito de desarrollo exclusivo es la carpeta `backend-node/` y los documentos
  específicos de la línea JavaScript en `docs/`.
- PROHIBICIÓN TOTAL: Tienes terminantemente prohibido crear, editar o eliminar
  archivos en `backend-django/`, `service-ai/`, `docker-compose.yml` o `.github/`.
  Esas áreas pertenecen al frente de Estefi.

## 2. Convenciones de Código en Node.js
- Stack: Node.js 20 LTS, Express, Mongoose 8, Socket.IO 4 y Zod.
- Arquitectura por capas desacopladas:
  * `src/models/`: Esquemas de Mongoose con validaciones e índices.
  * `src/controllers/`: Controladores livianos encargados de req/res.
  * `src/services/`: Lógica de negocio e integración HTTP con Django.
  * `src/middlewares/`: Manejo centralizado de errores y autenticación JWT.
- Integración resiliente: Toda llamada externa (Axios) hacia Django o hacia el
  microservicio de IA debe incluir un timeout estricto (máximo 2500 ms) y
  fallback para responder de modo degradado.

## 3. Política Estricta de Git (Cero Artefactos de IA)
- Prohibido agregar al staging area (`git add`) las carpetas `.specify/`,
  `.antigravity/` o cualquier archivo `*.prompt.md` o `*.agent.md`.
- Solo se versionan archivos de código de `backend-node/` y documentación en `docs/`.
```

Paso 2: Activar el Pre-Commit Hook en la Máquina de Ari

Para asegurar que su Git rechace automáticamente cualquier archivo de IA, Ari ejecuta en la raíz del proyecto (en Git Bash o terminal Linux/macOS):

```
cat << 'EOF' > .git/hooks/pre-commit
#!/bin/sh
FILES=$(git diff --cached --name-only | grep -E '^(\.specify/|\.antigravity/|.*\.prompt\.md|.*\.agent\.md)')
if [ -n "$FILES" ]; then
  echo "✖ ERROR: Archivos locales de desarrollo con IA detectados en el staging:"
  echo "$FILES"
  echo "Estos archivos están prohibidos en Git. Ejecuta 'git reset' antes de commitear."
  exit 1
fi
exit 0
EOF
chmod +x .git/hooks/pre-commit
```

(Si utiliza Windows PowerShell puro, simplemente debe verificar que el archivo `.git/hooks/pre-commit` contenga ese script).

4. Prompt Maestro para Antigravity IDE en la Computadora de Ari

Este es el prompt exacto que Ari debe copiar y pegar en su Antigravity IDE para comenzar a programar su entrega del TP 1:

```
Eres el agente de desarrollo para la línea JavaScript/Node.js del proyecto CineTrack
en la cátedra de Programación IV (UTN - FRRE).

Tu contexto de trabajo está estrictamente restringido a la carpeta `backend-node/`.
Tienes prohibido modificar `backend-django/` o `service-ai/`.

Misión: Implementar la API de Reseñas y Capa Social para el TP 1 cumpliendo con las
siguientes directivas:
```

```
1. Estructura de Proyecto en `backend-node/src/`:
   Crea la arquitectura organizada por capas:
   - `src/config/db.js`: Conexión asíncrona a MongoDB mediante Mongoose utilizando
     `process.env.MONGODB_URI`.
   - `src/models/Review.js`: Esquema Mongoose con:
     * `mediaItemId` (Number, requerido, indexado): ID de la serie/película en Django.
     * `userId` (String, requerido, indexado): UUID o ID del usuario.
     * `userName` (String, requerido).
     * `rating` (Number, min 1, max 10, requerido).
     * `comment` (String, max 2000 caracteres, requerido).
     * `isSpoiler` (Boolean, default false).
     * `replies` (Array de subdocumentos con `userId`, `userName`, `comment`,
       `createdAt`).
     * `createdAt` (Date, default Date.now).
     * Índices compuestos para optimizar búsquedas por `mediaItemId` y fecha.
   - `src/models/ActivityLog.js`: Esquema para auditoría de acciones del usuario.

2. Validaciones con Zod (`src/validators/reviewValidator.js`):
   - Esquema estricto para creación y actualización de reseñas (validar rango 1-10
     en rating y longitud de comentario).

3. Servicio de Integración con Django (`src/services/djangoService.js`):
   - Cliente Axios con `baseURL: process.env.DJANGO_API_URL` y
     `timeout: parseInt(process.env.DJANGO_TIMEOUT_MS) || 2500`.
   - Método `validateMediaExists(mediaId)`: Consulta `GET /media/{id}/` a Django.
   - Manejo de fallos: Si Django no responde o supera el timeout, no lanzar un
     error 500 fatal; registrar el warning y permitir la operación en modo
     degradado devolviendo un flag `{ available: false, degraded: true }`.

4. Controladores y Rutas (`src/controllers/reviewController.js` y
   `src/routes/reviewRoutes.js`):
   - CRUD completo:
     * `POST /api/v1/reviews/`: Crea una reseña (valida input con Zod y existencia
       en Django).
     * `GET /api/v1/reviews/media/:mediaId`: Lista reseñas con paginación
       (`page`, `limit`).
     * `POST /api/v1/reviews/:id/replies`: Agrega una respuesta anidada.
     * `DELETE /api/v1/reviews/:id`: Borrado lógico o físico.
   - Si la consulta opera en contingencia, incluir el encabezado HTTP
     `X-Degraded-Mode: true`.

5. Middleware de Errores y Logging:
   - Middleware centralizado en `src/middlewares/errorHandler.js` que formatee
     errores en JSON uniforme `{ error: { message, code, details } }`.
   - Middleware morgan o custom logger para registrar peticiones.

6. Pruebas Automatizadas con Jest:
   - Crear en `tests/review.test.js` 4 pruebas unitarias que verifiquen:
     * Creación exitosa de reseña.
     * Rechazo por validación Zod ante rating fuera de rango (ej. 15).
     * Inserción de respuesta anidada en el array de replies.
     * Comportamiento controlado cuando el servicio Django simula un timeout.

Implementa el código modularmente y déjalo listo para ejecutar con `npm run dev`
y `npm test`.
```

5. Flujo Operativo de Git para Ari (Paso a Paso Diario)

Para cumplir con la regla de la cátedra de cero pushes directos y revisiones por pares obligatorias:

```
1. Actualizar main:
   git checkout main
   git pull origin main

2. Crear rama de tarea:
   git checkout -b feature/node-reviews-crud
```

```
3. Trabajar con Antigravity en backend-node/

4. Confirmar cambios (solo su carpeta):
   git add backend-node/
   git commit -m "feat(node): implementar crud de resenas, esquemas mongoose y validacion zod"

5. Subir a GitHub:
   git push origin feature/node-reviews-crud

6. Abrir Pull Request en GitHub:
   - Base: main  <---  Compare: feature/node-reviews-crud
   - Asignar como Revisor a: BorghiEsteff (Estefi)

7. Estefi entra a GitHub, revisa el código, aprueba (Approve) y hace el Merge a main.
```

Con este esquema, cada una trabaja en un lenguaje y servicio independiente, Ari no satura el disco de su máquina, Antigravity asiste a ambas en sus respectivos frentes y el historial de Git refleja la colaboración equitativa exigida por la cátedra.